Capstone Project2

HEART DISEASE DETECTION

Team Members

UID1: T120200508 - Sharwari Kadam

UID2: T120200510 - Purab Sawant

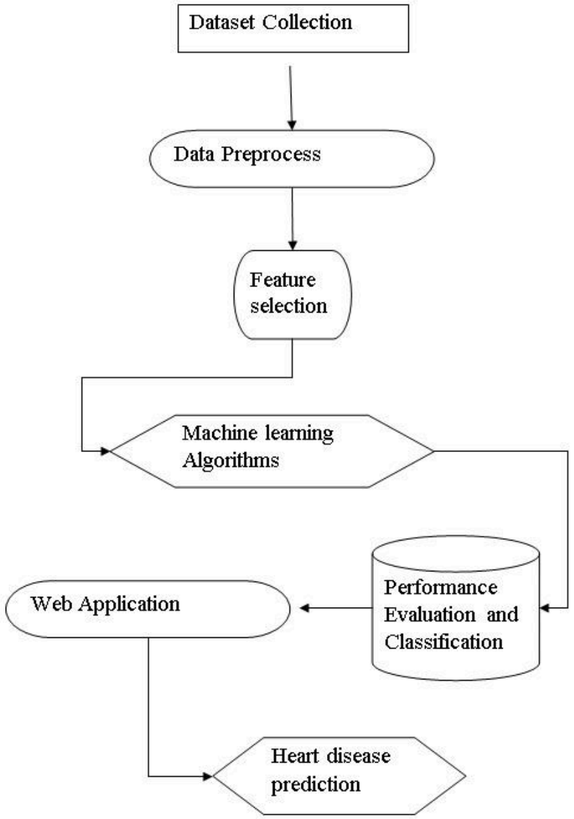
Table of Contents

- Intoduction
- Problem Statement
- Overview of the Project
- Data generation & Loading
- Data Exploration & Understanding
- Exploratory Data Analysis
- Data Preprocessing & Feature Engineering
- Model Building & Training
- Model Evaluation & Performance Analysis
- Stream API Deployment
- Docker Deployment
- Conclusion

INTRODUCTION

- Heart disease is one of the **leading causes of death worldwide**, making early detection extremely important
- Timely prediction can help in **preventing severe health conditions and reducing mortality rates**
- Traditional diagnosis methods can be **time-consuming and dependent on expert interpretation**
- This project uses **Data Analytics and Machine Learning** to predict heart disease based on patient data
- The dataset includes features like **blood pressure, cholesterol, ECG results, age, and exercise-induced angina**
- **Goal:** To build a predictive model that assists doctors in faster and more accurate decision-making

Can we accurately predict heart disease using patient diagnostic data?



Problem Statement

Primary Question:
Can we accurately predict the presence of heart disease in patients based on their diagnostic test results and clinical measurements?

Business Context:
Heart disease remains one of the leading causes of death globally. Early detection through diagnostic screening can significantly improve patient outcomes and reduce healthcare costs. This project simulates a real-world scenario where data scientists work with medical professionals to develop predictive models for clinical decision support.

Challenge:
Given a patient's diagnostic test results (blood pressure, cholesterol levels, ECG results, exercise capacity, etc.), predict whether the patient has heart disease or not.

Overview of the Project

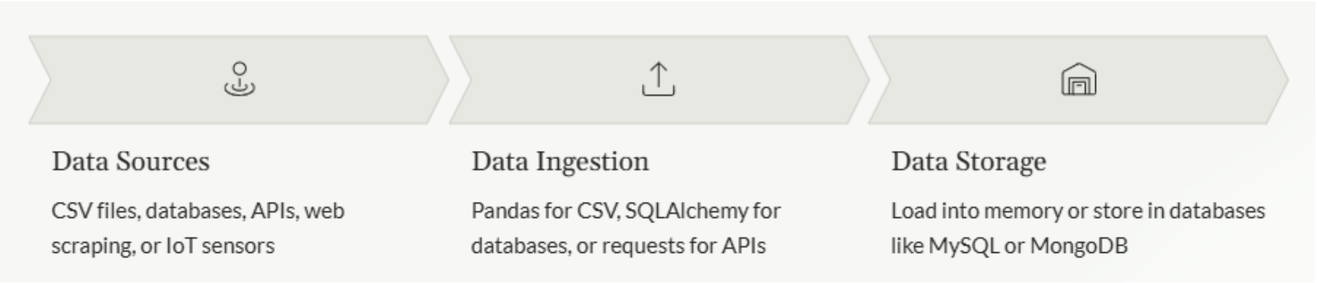
Objective:
Develop and optimize machine learning models to predict heart disease using clinical and diagnostic data.

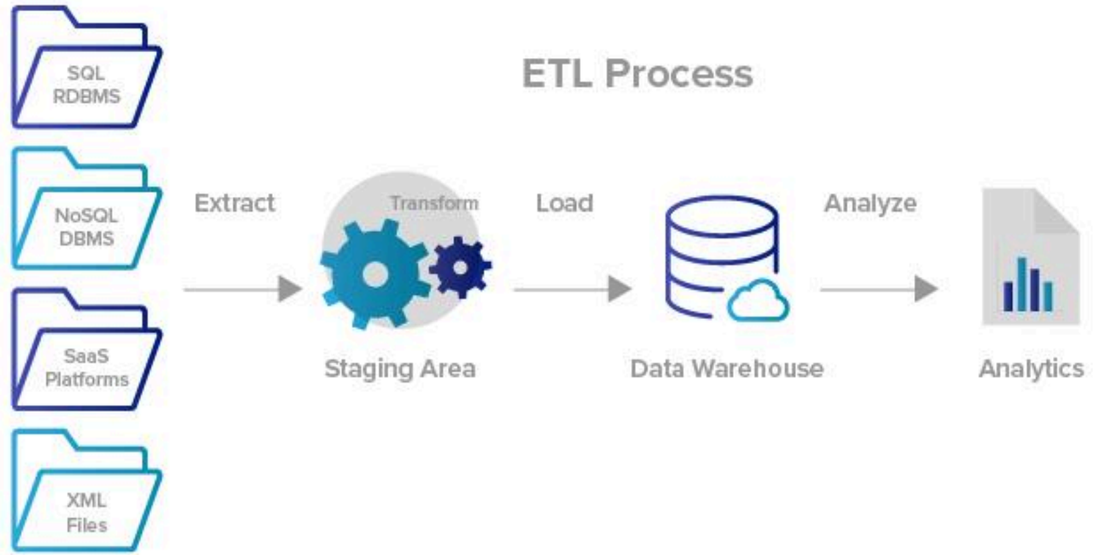
Scope:
Work with patient diagnostic test results (blood pressure, cholesterol, ECG, exercise capacity, etc.)
Build classification models to assist healthcare professionals in early detection and risk assessment

Approach:
Data Exploration & Preprocessing: Analyze, clean, and prepare data for modeling
Model Development: Implement multiple algorithms (Decision Tree, Random Forest, Logistic Regression, SVM)
Hyperparameter Optimization: Use grid search and cross-validation to improve model performance
Model Evaluation: Evaluate using accuracy, precision, recall, F1-score, ROC-AUC
Feature Analysis: Identify key indicators for heart disease risk

Outcome:
Accurate predictive models to support clinical decision-making and improve patient outcomes.

Data generation & Loading





- The dataset is sourced from **public clinical repositories for heart disease prediction**
- Contains **400 patient records with 13 clinical features** (age, sex, cholesterol, blood pressure, ECG results, etc.)
- Data is **clean and free from missing values**, ensuring smooth preprocessing
- Loaded using **Python Pandas library** for efficient data handling
- Initial inspection performed to verify **data types and consistency**
- This step ensures a *strong foundation* for **analysis and model building**

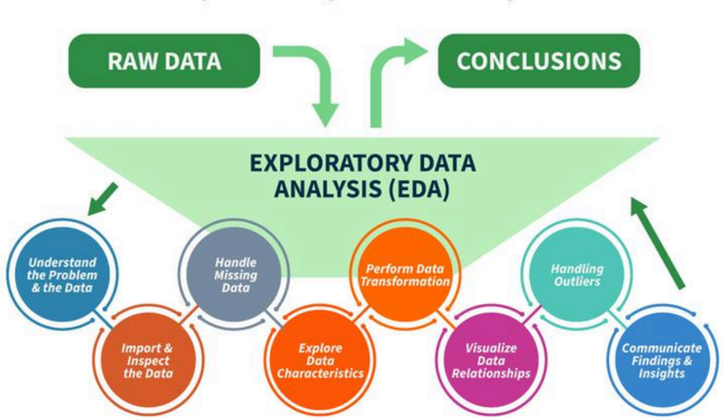
Data Exploration & Understanding

	age	sex	chest_pain_type	resting_blood_pressure	cholesterol	fasting_blood_sugar	resting_ecg	max_heart_rate	exercise_induced_angina	st_depression	st_slope	num_major_vessels	thalassemia
0	58	1	1	134	246	0	0	155	0	0.4	1	1	2
1	52	0	2	112	245	0	0	143	1	0.1	0	0	3
2	59	0	2	126	196	0	0	162	1	1.1	1	0	1
3	67	1	1	165	201	0	0	180	0	0.4	2	0	2
4	51	1	0	146	174	1	1	133	1	0.9	1	1	2

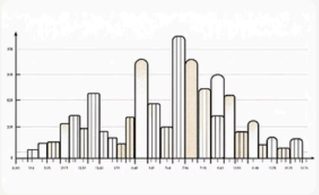
- The dataset contains **13 clinical features** such as age, sex, chest pain type, blood pressure, cholesterol, and ECG results
- Data shows **no missing values** and consistent ranges (e.g., age: 29–77, cholesterol: 120–564 mg/dl)
- Distribution analysis indicates **majority of patients are male**
- **Asymptomatic chest pain type** is the most frequent category
- Both **categorical and numerical features** are well distributed, enabling effective analysis
- Initial exploration helps in identifying **patterns and data quality before modeling**

Exploratory Data Analysis

- **Purpose:** Understand data patterns, relationships, and detect outliers
- **Target Variable Analysis:** Distribution of heart disease cases (0 = No, 1 = Yes)

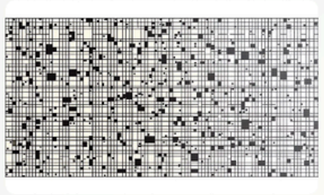


EDA transforms raw data into actionable insights through visualizations and statistical tests. This phase reveals correlations, trends, and anomalies that guide feature engineering.



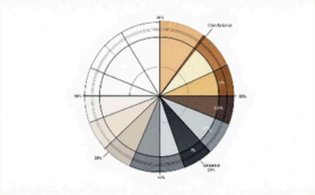
Distribution Analysis

Histograms and box plots reveal variable distributions and identify outliers



Correlation Study

Heatmaps show relationships between variables, guiding feature selection



Class Balance

Pie charts and bar graphs show target variable distribution

Data Preprocessing & Feature Engineering

- **Missing Values Handling:** Dataset contains no missing values, ensuring clean data
- **Encoding:** Converted categorical features (chest pain, ECG, thal, etc.) into numerical format using encoding techniques
- **Feature Scaling:** Standardized numerical features (cholesterol, blood pressure, max heart rate) for uniform range
- **Data Splitting:** Divided dataset into 80% training and 20% testing sets
- **Feature Selection:** Selected important features based on correlation analysis to improve model performance

01

Data Cleaning

Handle missing values through imputation or deletion, remove duplicates, fix inconsistent entries

02

Feature Encoding

Convert categorical variables using one-hot encoding, label encoding, or target encoding

03

Feature Scaling

Normalize or standardize numerical features for algorithms sensitive to scale

04

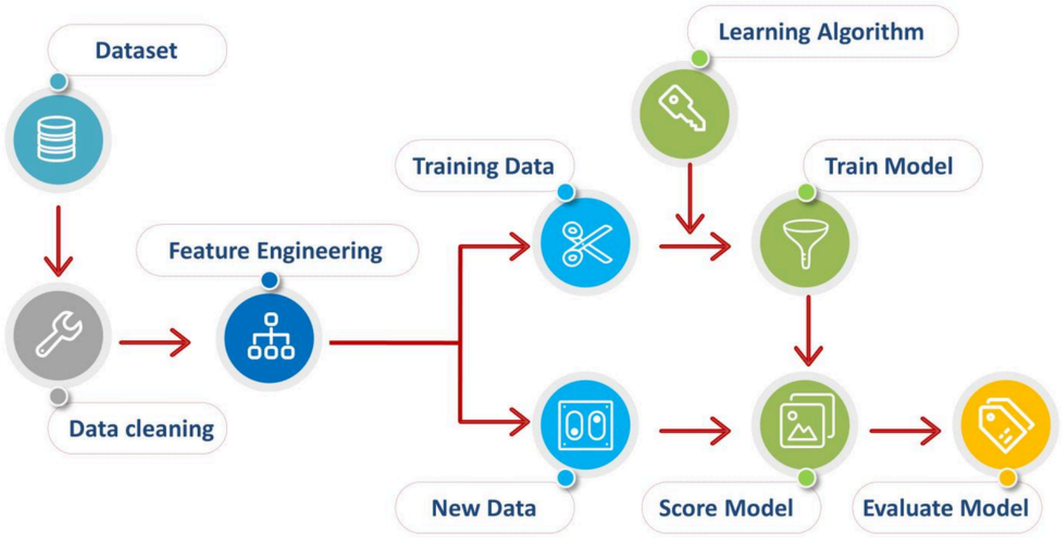
Feature Creation

Generate new features from existing data, such as date components or interaction terms

05

Dimensionality Reduction

Apply PCA or feature selection to reduce variables while preserving information



Model Building & Training

TNS INDIA FOUNDATION
Partners in Economic Transformation

Model Selection

- Logistic Regression, Decision Tree
- Random Forest, SVM
- Compared multiple models for best performance

Step 1

Training

- Split: 80% Train / 20% Test
- X: Patient features | y: Target (0/1)
- Models trained to learn patterns

Step 2

Prediction

- y_pred → Class prediction
- y_prob → Probability score
- Enables risk-based decision making

Step 3

Optimization

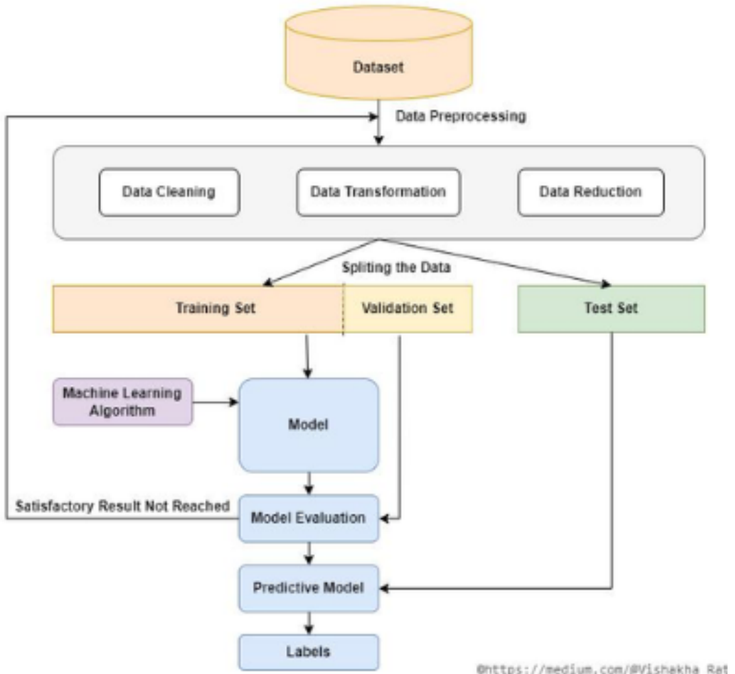
- Used GridSearchCV
- Tuned parameters for better accuracy
- Improved model performance

Step 4

Model Comparison

- Compared results of all models
- Selected best-performing model

Step 5



@https://medium.com/@Vishakha_Rat

Model Evaluation & Performance Analysis

Step 1: Evaluation Metrics

- *Accuracy:* Overall correct predictions
- *Precision:* Correct positive predictions
- *Recall:* Actual positives identified
- *F1-Score:* Balance of Precision & Recall
- *ROC-AUC:* Model's ability to distinguish classes

Step 2: Confusion Matrix

- Shows TP, TN, FP, FN
- Helps analyze model errors and performance

Step 3: Feature Importance

- Identified key features using Decision Tree & Random Forest
- Important features: Chest pain, Max heart rate, Cholesterol

Step 4: Performance Summary

- Compared all models using above metrics
- Selected best model based on ROC-AUC & F1-Score

Stream API Deployment

Step 1: Application Development

- Built interactive web app using Streamlit (Python)
- Created UI in app.py for user input and prediction

Step 2: Running the App

- Executed using: *streamlit run app.py*
- App runs locally in browser

Step 3: Deployment

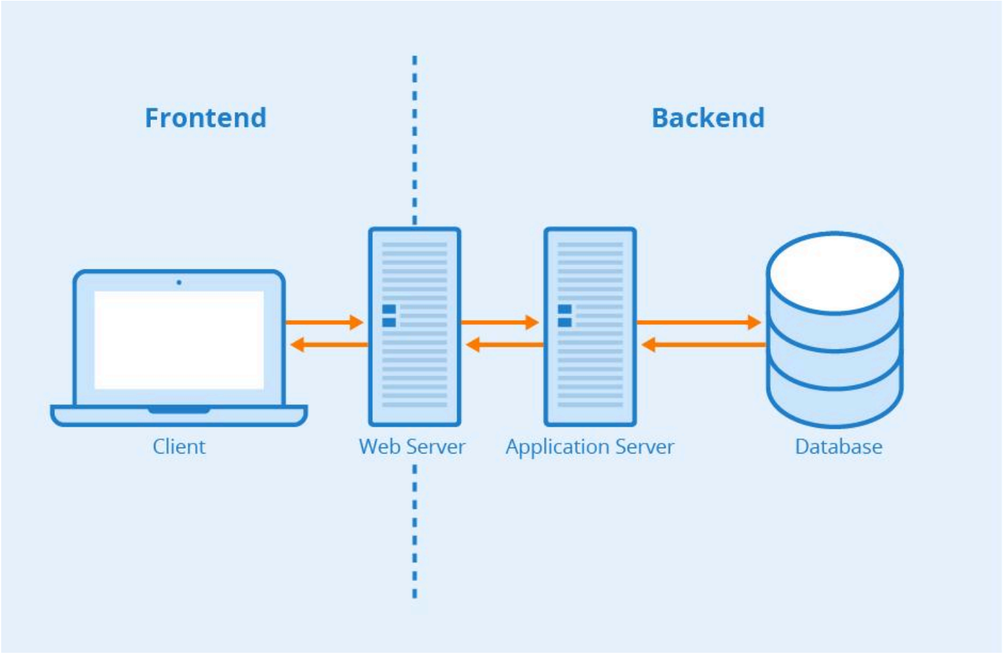
- Deployed on platforms like:
  - Streamlit Cloud
  - Heroku / AWS / GCP

Advantages

- Fast and easy deployment
- No separate frontend required

Limitations

- Limited UI customization
- Not suitable for large-scale production APIs



Docker Deployment

Step 1: Containerization

- Packaged application + dependencies using Docker
- Ensures consistent runtime environment

Step 2: Build & Run

- Created Dockerfile
- Commands used: docker build → Create image
  docker run → Run container

Step 3: Deployment

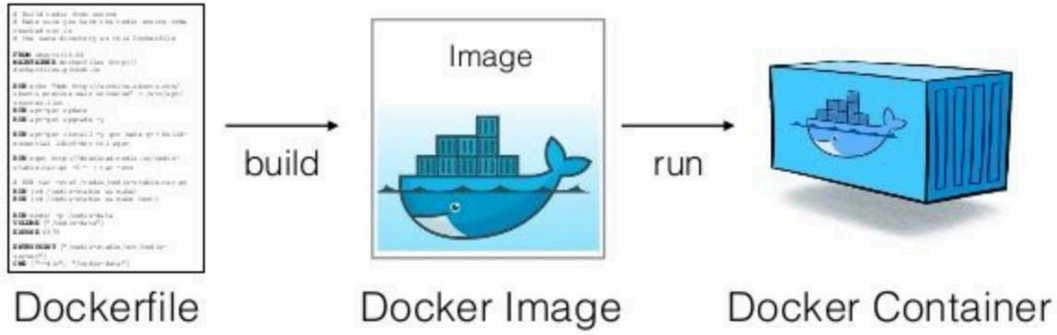
- Deployed via: Docker Hub
  AWS / GCP / Azure
  Kubernetes (for scaling)

Advantages

- Portable across systems
- Scalable and reliable

Limitations

- Initial learning curve
- Requires orchestration for large-scale deployment



Conclusion

- Machine learning techniques can effectively predict heart disease using clinical and diagnostic data
- The project achieved reliable performance by applying and comparing multiple classification models
- Features like cholesterol, chest pain type, and maximum heart rate were found to be highly influential in prediction
- The model enables early detection of high-risk patients, improving chances of timely treatment
- This system can act as a decision-support tool for doctors, enhancing efficiency and accuracy in diagnosis

This project demonstrates the potential of data-driven solutions in transforming healthcare through early and accurate disease prediction.